MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

PROJECT REPORT

of

CSS 2213 : OBJECT - ORIENTED SOFTWARE DEVELOPMENT LAB

Multi-Cryptocurrency Trading Simulator

A JavaFX-Based Desktop Application with OOP, Threading & File I/O

Submitted by

Rajula Sai Chetan Reddy

240905164

Roll no.26

CSE - A

School of Computer Engineering

Manipal Institute of Technology, Manipal.

April 2026

1. Introduction

Cryptocurrency markets are highly dynamic and operate continuously, making them well suited for simulation-based learning tools. The Multi-Cryptocurrency Trading Simulator is a desktop application developed using Java and the JavaFX graphical user interface framework. It allows users to practice buying and selling five major cryptocurrencies including Bitcoin (BTC), Ethereum (ETH), Solana (SOL), Dogecoin (DOGE), and Cardano (ADA) in a risk-free simulated environment.

The application demonstrates key Object-Oriented Programming concepts while also incorporating advanced Java features such as multithreading, file serialization, and collections. Users can create individual profiles, perform trades based on either monetary value or coin quantity, track portfolio performance in real time, and view detailed transaction histories. All user data is stored using Java object serialization, allowing it to persist across sessions.

This project was developed as part of the OOSDL course with the aim of building a functional and well-structured Java desktop application. The complete implementation, including data models and the JavaFX user interface, is contained in a single file named App7.java.

2. Objectives

- The primary objective of this project is to design and implement a multi-user cryptocurrency trading simulator using Java and JavaFX.
- The project aims to demonstrate core Object-Oriented Programming principles such as abstraction, encapsulation, inheritance, and polymorphism through practical class design.
- It also focuses on implementing multithreading to handle concurrent live price updates using the Thread class and synchronization techniques.
- Another objective is to store and restore application data across sessions using Java object serialization.
- The project further applies concepts from the Java Collections Framework, including the use of HashMap, ArrayList, and Iterator, along with wrapper classes and autoboxing and unboxing.
- In addition, it aims to develop a clean and user-friendly graphical interface using JavaFX, featuring multiple scenes, tab panes, radio buttons, and alert dialogs.
- Finally, the application provides a portfolio dashboard that displays live unrealized profit and loss along with historically realized profit and loss for each user.

3. Methodology

The application was developed step by step using a layered structure, starting from the data model and moving up to the graphical interface.

3.1 Architecture and Class Design

The system is organized into multiple classes, each handling a specific responsibility:

- ***Asset*** (abstract): Base class that defines common properties for all tradable assets.
- ***Crypto***: Concrete subclass of Asset. Stores the live price with a volatile field and synchronized setter for thread safety.
- ***Portfolio***: Encapsulates a user's holdings in a single cryptocurrency - quantity, total invested, average cost, and booked P&L. Implements Serializable for persistence.
- ***User***: Holds a user's name, a HashMap of Portfolio objects, and a HashMap of ArrayLists for transaction history.
- ***TradingService***: Provides synchronized buy and sell methods that coordinate updates to Portfolio and transaction logs. Prevents race conditions.
- ***PriceUpdater***: Extends Thread. Each cryptocurrency gets its own PriceUpdater daemon thread, simulating price volatility with random increments..
- ***FileHandler***: Handles serialization and deserialization of the entire users map to/from a binary file (crypto_data.dat).
- ***App7*** (main class): JavaFX Application subclass. Manages the GUI scenes - Login and Trading.

3.2 OOP Concepts Applied

The project applies key OOP concepts as follows:

- **Abstraction** is achieved through the abstract class *Asset*, which defines a common method for all tradable assets. The *Crypto* class provides the actual implementation of this method.
- **Inheritance** is demonstrated by the *Crypto* class, which extends the *Asset* class and reuses its structure while adding its own properties such as name and symbol.
- **Encapsulation** is followed in the *Portfolio* and *User* classes, where internal data such as quantity and total investment is kept private and accessed only through methods.
- **Polymorphism** is used by handling different cryptocurrency objects through references of the *Asset* type, allowing the same method to work with different objects.
- **Multithreading** is implemented using the *PriceUpdater* class, where separate threads update cryptocurrency prices at regular intervals.
- **Synchronization** is applied in trading operations to ensure that multiple threads do not cause conflicts when buying or selling assets.
- **Serialization** is used to store and retrieve user data. Classes such as *User* and *Portfolio* implement Serializable, and data is saved using object streams.
- **HashMap** is used to store user portfolios and to map cryptocurrency names to their respective objects.
- **ArrayList** is used to maintain the transaction history for each user.
- **Iterator** is used to traverse collections while displaying data in the interface.
- **Wrapper classes and autoboxing/unboxing** are used when storing numeric values in collections, allowing primitive data types to be handled as objects.

3.3 Price Simulation Engine

Each cryptocurrency is assigned a volatility coefficient to simulate realistic price movement (for example, Bitcoin ±Rs. 500 per tick and Dogecoin ±Rs. 0.003 per tick). The *PriceUpdater* thread generates a random price change within this range at fixed intervals of two seconds. It then updates the price using the synchronized *setPrice()* method to ensure thread safety. After updating, the UI is refreshed using *Platform.runLater()* so that changes are reflected on the JavaFX Application Thread. Each *PriceUpdater* thread is set as a daemon thread using *setDaemon(true)*, ensuring that these background threads do not prevent the JVM from exiting when the application is closed.

3.4 Portfolio Accounting

The *Portfolio* class maintains a weighted average cost basis for each cryptocurrency. During a BUY operation, both the quantity and total invested amount are added to the existing values. During a SELL operation, the average cost is calculated from current holdings to determine the cost basis of the sold quantity. The difference between the selling price and this cost basis is recorded as *bookedPL* (realized profit or loss). The user interface displays both unrealized P&L, calculated as the difference between current market value and total invested amount for active holdings, and booked P&L, which represents profit or loss already realized through completed transactions. These values are shown for each cryptocurrency as well as for the overall portfolio.

3.5 Data Persistence

The *FileHandler* class handles serialization and deserialization of application data. The complete *allUsers* HashMap is written to a file named *crypto_data.dat* using *ObjectOutputStream* when the user selects Save Data or Logout. On application startup, the system attempts to read this file using *ObjectInputStream* to restore previously saved data. If the file is not found or cannot be read due to corruption, the application initializes with an empty HashMap. This approach ensures that user portfolios and transaction histories are preserved across sessions.

3.6 Graphical User Interface

The graphical interface is developed using JavaFX and is divided into two main scenes.

- **Login Scene**: Displays all registered users as selectable buttons. A dialog box allows new users to be added. User data is stored in the serialized HashMap and identified by unique names.
- **Trading Scene**: Provides the main functionality of the application. A sidebar contains options for selecting cryptocurrencies and accessing features such as All Transactions, Save Data, and Logout. The main area displays the selected cryptocurrency's live price, along with a trading panel that uses RadioButton controls to switch between trading by monetary amount or coin quantity. A TabPane is used to organize information into a Portfolio Summary tab, showing a complete view of all holdings, and a Per-Coin Transactions tab, displaying detailed transaction history for each cryptocurrency.

4. Results and Discussion

4.1 Application Screenshots

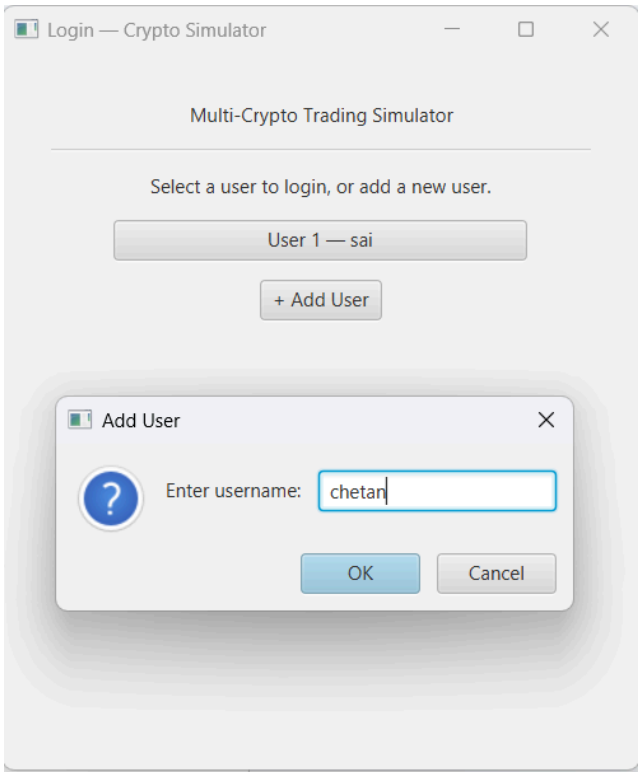

Figure 1: Login Scene - User selection and Add User dialog

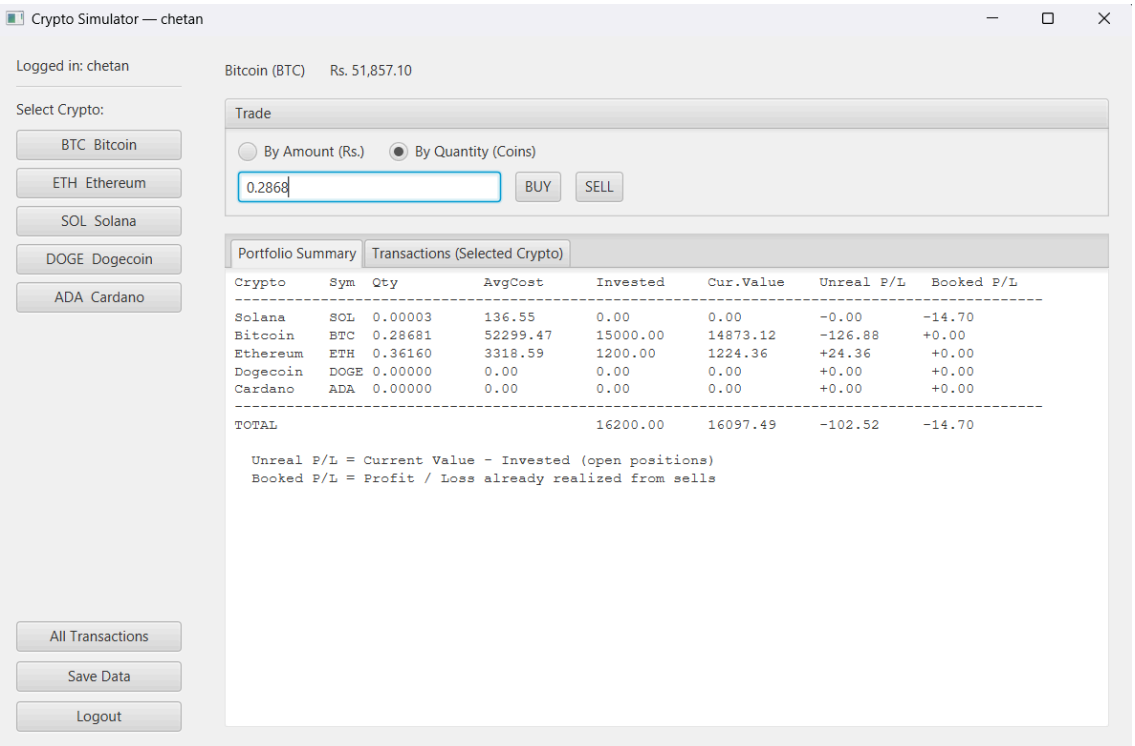

Figure 2: Trading Scene - Portfolio Summary tab showing live P&L across all cryptocurrencies

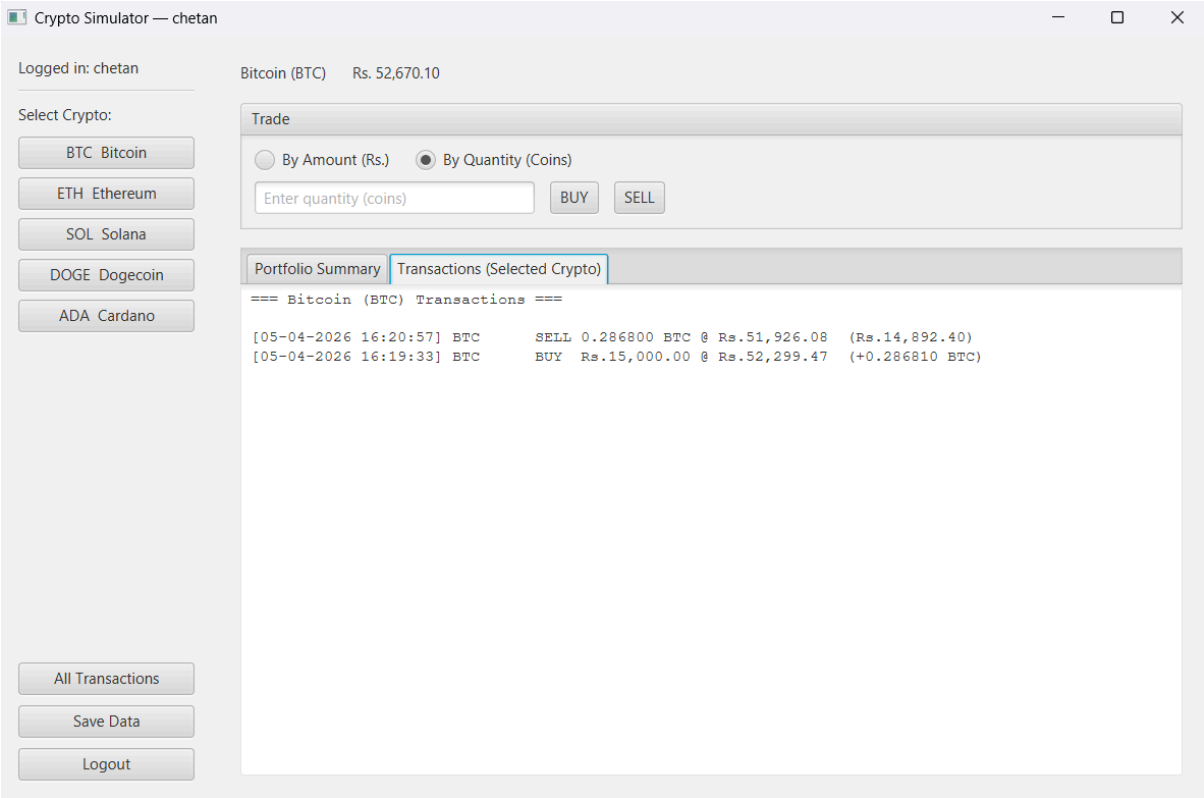

Figure 3: Transactions (Selected Crypto) tab - Bitcoin buy and sell history in reverse chronological order

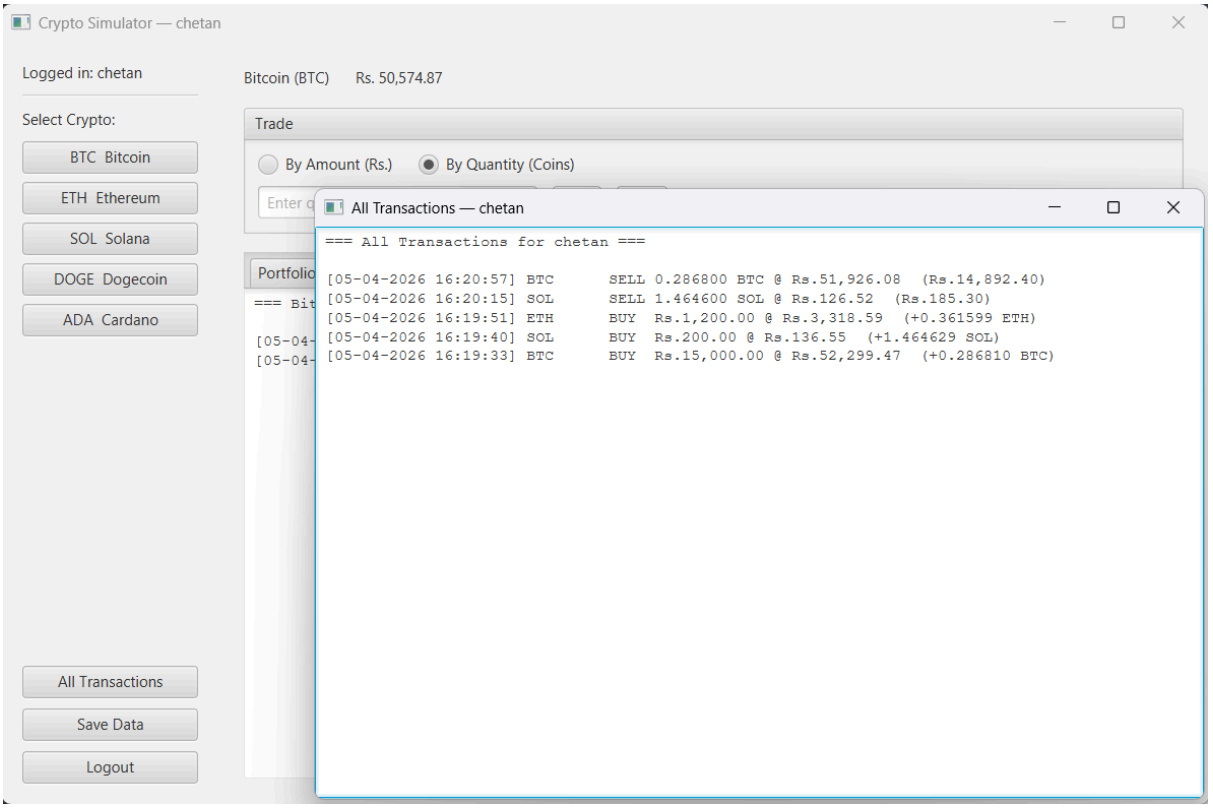

Figure 4: All Transactions window - Combined trade history across all cryptocurrencies for user 'chetan'

4.2 Outcomes

The application successfully meets all the stated objectives. The following features have been implemented and verified:

- User registration and login with persistent profile storage.
- Live price simulation for five cryptocurrencies using separate daemon threads.
- Buy and sell operations in both rupee-amount mode and coin-quantity mode.
- A real-time portfolio dashboard displaying both unrealized and realized profit and loss.
- Transaction history views for individual cryptocurrencies as well as all transactions, displayed in reverse chronological order.
- Complete data persistence, ensuring that portfolios and transaction histories are retained after restarting the application.
- Error handling for invalid inputs, such as non-numeric values or attempting to sell more coins than available, using JavaFX Alert dialogs.

4.3 OOP Design Strengths

The use of an abstract *Asset* class as the base for *Crypto* makes the design extensible. Additional asset types, such as stocks or commodities, can be introduced by creating new subclasses without changing the existing *TradingService* or *Portfolio* logic. The separation of *TradingService* from *Portfolio* helps keep business logic independent of the data model, following the single responsibility principle. Synchronized methods in both *TradingService* and *Crypto* ensure thread safety by preventing issues that could arise from concurrent price updates.

4.4 Observations

During testing, cryptocurrency prices updated consistently every two seconds in the user interface. Portfolio values were refreshed immediately after each trade. Realized profit and loss accumulated correctly over multiple sell operations, and the weighted average cost was recalculated accurately after partial sales. Data stored in *crypto_data.dat* was successfully loaded when the application was restarted, with complete transaction history preserved.

4.5 Limitations

- Prices are generated through simulation and do not reflect real market data.
- No authentication mechanism is implemented, allowing any user on the system to access existing profiles.
- The interface does not include advanced visualizations such as candlestick charts or historical price graphs.

5. Conclusion

The Multi-Cryptocurrency Trading Simulator demonstrates a wide range of Java programming concepts within a single, well-structured application. It brings together Object-Oriented Programming concepts such as abstraction, inheritance, encapsulation, and polymorphism, along with concurrent programming using Thread, synchronized methods,

volatile variables, and Platform.runLater(). It also incorporates the Java Collections Framework, including HashMap, ArrayList, and Iterator, along with wrapper classes, autoboxing and unboxing, and Java I/O through serialization using ObjectOutputStream and ObjectInputStream. All of these components are integrated into a functional JavaFX-based graphical application.

The application provides practical value by allowing users to observe how price changes affect their portfolio in real time, make trading decisions, and review transaction history. The use of data persistence ensures that user data is retained across sessions, making the system consistent and reliable in everyday use.

Future improvements can include integration with a live cryptocurrency price API, addition of password-based authentication for users, implementation of price history charts using JavaFX LineChart, and the use of a cloud-based backend to enable access across multiple devices. The current design, with clearly separated classes and responsibilities, allows these features to be added without major changes to the existing structure.

References

1. https://docs.oracle.com/en/java/ - Oracle Corporation. (2024). Java SE 21 Documentation
2. https://openjfx.io/javadoc/21/ - Oracle Corporation. (2024). JavaFX 21 API Documentation
3. Goetz, B., et al. (2006). Java Concurrency in Practice. Addison-Wesley Professional.